

# Booleans and Comparisons

Truth values, comparisons, and explicit conversions

---

Gregory S. DeLozier, PhD

# Breaking Changes

**New keywords:** `true`, `false`, `boolean`, `type`

Rename earlier variables that use those exact names.

`true_value`, `boolean_value`, and `typewriter` remain identifiers.

# Computing an Answer

```
__input = "21";  
answer = number(input()) * 2;  
large_enough = answer >= 40;  
print(large_enough);
```

```
true
```

# A New Literal Node

```
ready = true;
```

```
{"tag": "boolean", "value": True}
```

Vertex spelling: true. Python value: True.

# Comparing Values

| Expression                  | Result |
|-----------------------------|--------|
| <code>42 == 42.0</code>     | true   |
| <code>"dog" != "cat"</code> | true   |
| <code>-1.5 &lt; .5</code>   | true   |
| <code>2 &lt;= 2</code>      | true   |
| <code>2 &gt; 3</code>       | false  |
| <code>3 &gt;= 3</code>      | true   |

# Types Still Matter

| Expression                    | Result     |
|-------------------------------|------------|
| <code>true == 1</code>        | false      |
| <code>"1" == 1</code>         | false      |
| <code>"a" &lt; "b"</code>     | Type error |
| <code>true + 1</code>         | Type error |
| <code>number(true) + 2</code> | 3          |

# Explicit Conversions

| Expression                      | Value                |
|---------------------------------|----------------------|
| <code>number(true)</code>       | <code>1</code>       |
| <code>string(false)</code>      | <code>"false"</code> |
| <code>boolean(-2)</code>        | <code>true</code>    |
| <code>boolean(0)</code>         | <code>false</code>   |
| <code>boolean(" FALSE ")</code> | <code>false</code>   |

# Text Needs an Interpretation

|                                   |                          |
|-----------------------------------|--------------------------|
| <code>boolean("false")</code>     | <code>false</code>       |
| <code>boolean("1")</code>         | <code>value error</code> |
| <code>boolean(number("1"))</code> | <code>true</code>        |

**No nonempty-string truthiness.**

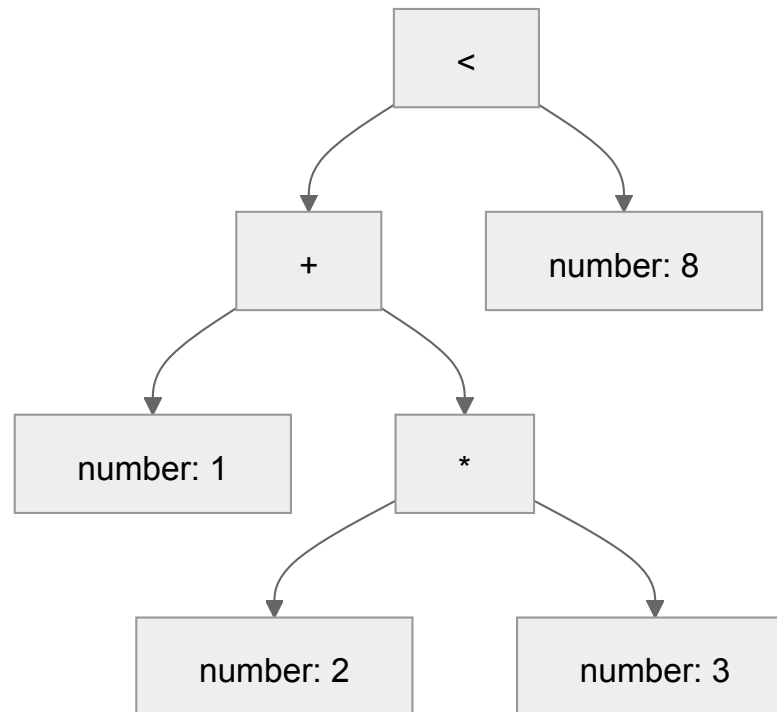
# The Type of a Value

| Expression                  | Result                 |
|-----------------------------|------------------------|
| <code>type(42)</code>       | <code>"number"</code>  |
| <code>type(.5)</code>       | <code>"number"</code>  |
| <code>type("42")</code>     | <code>"string"</code>  |
| <code>type(1 &lt; 2)</code> | <code>"boolean"</code> |

`type(input())` still reads input.

# The Comparison Tree

1 + 2 \* 3 < 8



# One Comparison at a Time

```
expression ::= comparison
comparison ::= arithmetic_expression
              [ compare_op arithmetic_expression ]
```

| Expression      | Result       |
|-----------------|--------------|
| 1 < 2 < 3       | Syntax error |
| (1 < 2) == true | true         |
| (1 < 2) < 3     | Type error   |

# Checking the Implementation

```
bash run-tests.sh
```

All six comparisons. Exact result types. Conversion boundaries.

**Input arguments run once; comparison operands run left to right.**

# Practice

```
true == 1  
number(true) == 1  
boolean("0")  
boolean(number("0"))
```

**Which succeed, and what does each produce?**